TP11 — Organisation du moteur de bataille

Programmation Java • Projet fil rouge : RPG tactique au tour par tour

Objectif général. Transformer la simulation du TP10 en un projet mieux organisé : les équipes, les états et les phases de bataille ne doivent plus dépendre exclusivement de Main. Le comportement obtenu à la fin du TP10 doit être préservé.

Pré-requis. TP10 terminé ; classes Unite, Epeiste, Cavalier, Mage, Clerc, interface Soigneur et enum EtatUnite disponibles.

Règle de travail. Un seul projet Java évolutif, aucun fichier Exercice1.java, etc. Modifier et enrichir les mêmes classes à chaque exercice. Chercher les méthodes de la bibliothèque Java dans la documentation si nécessaire. Ne pas utiliser d'IA pour générer le code des exercices.

Périmètre. Exécution en console ; pas de grille, d'interface JavaFX, de déplacement, d'intelligence artificielle, de réseau ni de moteur automatique fondé sur l'enum Action.

Consignes communes

• Préserver les personnages et caractéristiques du TP10 : Chrom 24 PV/force 8 ; Frederick 30 PV/défense 7 ; Robin 22 PV/magie 9 ; Lissa 18 PV ; Vaike 25 PV/force 6.
• Conserver la distinction entre l'équipe alliée et l'équipe ennemie.
• Éviter la duplication de logique et les accès directs aux attributs privés.
• Afficher les résultats des vérifications demandées ; les méthodes métier ne doivent pas être obligées d'imprimer dans le terminal.
• Enregistrer chaque exercice validé dans un commit Git.

Exercice 1 — Créer la classe Partie

Travail demandé. Créer une classe Partie chargée de conserver les équipes alliée et ennemie. Son constructeur reçoit les deux collections d'unités. Ajouter des méthodes permettant de récupérer chacune des équipes. Depuis Main, créer une partie avec les cinq personnages existants.

Contrainte. Ne pas recréer les personnages dans Partie. Ne pas supprimer les sous-classes de Unite. Choisir des types de paramètres adaptés aux collections.

Validation. Une partie contient quatre alliés et un ennemi ; leur ordre d'affichage est conservé.

Exercice 2 — Centraliser les états d'activation

Travail demandé. Déplacer dans Partie la gestion de la collection associant chaque unité à son EtatUnite. À la création d'une partie, initialiser à DISPONIBLE les unités des deux équipes. Prévoir une opération permettant de consulter l'état d'une unité.

Contrainte. Associer les états aux objets Unite, jamais aux indices. Ne pas maintenir deux copies indépendantes des états dans Main et Partie.

Validation. Dans Main, les cinq unités ont l'état DISPONIBLE au début de la partie.

Exercice 3 — Déplacer la vérification peutAgir

Travail demandé. Créer dans Partie une opération peutAgir recevant une unité. Elle renvoie vrai uniquement lorsque cette unité possède des PV strictement positifs et qu'elle est DISPONIBLE. Refaire les trois tests du TP10 : disponible, terminée, puis à 0 PV.

Contrainte. La méthode n'affiche rien, ne change ni les PV ni les états, et s'appuie sur la gestion d'états de Partie. Rétablir les données de test ensuite.

Validation. Les trois résultats, dans cet ordre, sont true, false, false.

Exercice 4 — Déplacer l'action attendre

Travail demandé. Créer dans Partie une opération attendre recevant une unité et retournant si l'action a réussi. Tester l'attente de Frederick, puis une seconde tentative immédiate. Réinitialiser les données de test avant la suite.

Contrainte. Une unité ne peut attendre que si elle peut agir. Après une attente réussie, son état devient TERMINEE ; les PV ne changent pas.

Validation. Première attente : true ; deuxième attente : false ; PV de Frederick : 30 / 30.

Exercice 5 — Encapsuler l'attaque rapide

Travail demandé. Créer dans Partie une opération permettant à un Epeiste d'attaquer une Unite avec attaqueRapide(). La méthode vérifie l'autorisation d'agir et termine l'activation uniquement en cas de réussite. Depuis Main, faire attaquer Chrom contre Vaike deux fois de suite.

Contrainte. Réutiliser attaqueRapide et peutAgir. Ne pas refaire le calcul des dégâts ni modifier les PV directement. Ne pas appliquer une attaque si elle a été refusée.

Validation. Première attaque acceptée : Vaike 17 / 25 PV ; deuxième tentative refusée : Vaike reste à 17 / 25 PV.

Exercice 6 — Encapsuler le soin

Travail demandé. Ajouter à Partie une opération de soin recevant une unité qui sait soigner, une cible et une quantité de PV. Pour le scénario, Chrom subit 6 dégâts, puis Lissa soigne Chrom de 5 PV ; une seconde activation de Lissa est refusée.

Contrainte. Utiliser l'interface Soigneur pour effectuer le soin ; l'état de Lissa doit être vérifié et modifié. Ne pas recopier le plafonnement des PV de Unite.soigner.

Validation. Après dégâts et soin : Chrom 23 / 24 PV ; Lissa est TERMINEE ; second soin non appliqué.

Exercice 7 — Fin de phase par équipe

Travail demandé. Créer dans Partie une méthode qui vérifie si toutes les unités d'une équipe passée en argument ont l'état TERMINEE. Faire attendre Frederick et Robin, puis vérifier la fin de la phase alliée avant et après le soin de Lissa.

Contrainte. Ne pas supposer qu'il y a toujours quatre alliés. Les états ennemis ne doivent pas intervenir dans la vérification de l'équipe alliée.

Validation. Avant Lissa : false ; après Lissa : true. Vaike peut être encore DISPONIBLE.

Exercice 8 — Simuler la phase ennemie

Travail demandé. Après la phase alliée, faire attaquer Vaike sur Chrom par le mécanisme d'attaque de Partie, puis contrôler la fin de la phase ennemie. L'ennemi joue une seule fois.

Contrainte. Ne pas réinitialiser les alliés au début de la phase ennemie. Ne pas développer d'IA ni sélectionner automatiquement les cibles.

Validation. Chrom : 17 / 24 PV ; Vaike : 17 / 25 PV ; phase ennemie terminée : true.

Exercice 9 — Préparer un nouveau tour

Travail demandé. Créer dans Partie une opération qui réinitialise à DISPONIBLE les unités vivantes d'une équipe. Après les deux phases, réinitialiser les deux équipes et afficher les états. Pour tester la condition, mettre temporairement Frederick à 0 PV après son action, puis observer son état.

Contrainte. Ne pas restaurer les PV, ne pas recréer les objets et ne pas remettre DISPONIBLE une unité à 0 PV. Le second tour ne doit pas être simulé.

Validation. Les unités vivantes redeviennent DISPONIBLE ; Frederick, à 0 PV, conserve son dernier état. Chrom reste à 17/24 PV et Vaike à 17/25 PV.

Exercice 10 — Alléger Main et présenter la bataille

Travail demandé. Réorganiser Main pour que l'enchaînement des opérations soit lisible : création des personnages, construction de Partie, phase alliée, phase ennemie, préparation du tour suivant. Conserver les tests ponctuels des exercices précédents uniquement s'ils ne perturbent pas le scénario principal, ou les isoler clairement. Produire un affichage lisible avec titres de phases et bilan final.

Contrainte. Ne pas ajouter de nouvelle mécanique de jeu. Main orchestre le scénario ; Partie porte la logique des états et des actions. Éviter de dupliquer les mêmes vérifications dans les deux classes.

Validation. La simulation conserve les résultats attendus : Vaike 17/25 après l'attaque alliée ; Chrom 23/24 après soin, puis 17/24 après attaque ennemie ; fin des deux phases : true ; nouveau tour préparé.

Bilan attendu

À la fin de ce TP, tu possèdes une première classe Partie capable de gérer les équipes, les états et plusieurs actions. Le fichier Main.java sert principalement à décrire la succession des événements. Ce sera la base des prochains TP.

Livrables : code Java compilable ; dépôt GitHub à jour ; aucune solution rédigée dans ce sujet.